

Encoding, normalización y parsers

Objetivos

Distinguir representación de significado, ordenar transformaciones y explicar por qué dos componentes pueden interpretar los mismos bytes de modo diferente.

Encoding no es cifrado

Base64, percent-encoding, Unicode y escapes JSON representan datos; no prueban confidencialidad. Decodificar recupera otra representación sin secreto. El significado aparece cuando un parser consume el resultado.

Ejemplo conceptual:

```
entrada HTTP:      %2e%2e%2fconfig
decodificación URL: ../config
normalización ruta: /srv/config
operación final:    open('/srv/config')
```

Bloquear la primera cadena y ejecutar la tercera es validación en una fase incorrecta.

Orden de procesamiento

Documenta siempre:

```
bytes -> decodificación de transporte -> parser de formato -> coerción de tipo
      -> normalización/canonicalización -> validación -> parser final -> sink
```

No presupongas el orden. Una aplicación puede validar antes de decodificar, decodificar dos veces o delegar la normalización al sistema de ficheros.

Parser, intérprete y sink

- Un parser convierte una representación en estructura: JSON a objetos, URL a componentes, SQL a árbol sintáctico.
- Un intérprete asigna comportamiento a esa estructura: shell, motor SQL o plantilla.
- Un sink es una operación sensible: ejecutar, consultar, abrir fichero, realizar petición o decidir acceso.

La vulnerabilidad aparece cuando datos controlados alteran estructura o selección de recurso en el sink sin una frontera segura.

Validación textual frente a semántica

"127.0.0.1" no aparece es una comparación textual. la dirección resuelta no pertenece a loopback, privada ni metadatos es una condición semántica. La segunda debe aplicarse al destino final y repetirse tras redirecciones o cambios de resolución cuando corresponda.

Cómo diagnosticar

1. Parte de una entrada válida.
2. Cambia una sola representación.
3. Observa errores de capa: URL, JSON, plantilla, SQL, shell o filesystem.
4. Compara lo enviado con lo registrado y lo ejecutado cuando el laboratorio ofrece logs.
5. Determina cuántas decodificaciones ocurren.
6. Confirma el significado final con una evidencia inocua.

Error frecuente

Una variante codificada que funciona no demuestra por sí sola que "el filtro se saltó". Puede haber cambiado el transporte, el router o la selección de recurso. La conclusión debe señalar qué componente comparó qué forma.

Referencias

- [RFC 3986, percent-encoding y normalización](#)
- [Fuente interna: filtros incompletos](#)

Anterior: [Redes](#). Siguiendo: [Linux y shell](#).